Unary power covering at PA strength

Eliminating the source parameter, with a division-free positive proof-search interface

Research prepared for Vladimir Reshetnikov

27 October 2026

Abstract

The eight supplied drafts already establish a compact equality-free power-covering theory whose matrix may mention its tested object and its source. We remove the source as well. For every formula $\varphi(x)$ with no other free variable, the sole schema asserts that each source a has a cover containing every φ-object whose members lie in a, and containing only φ-objects. There are no additional nonlogical axioms. We prove that this unary theory and full Peano arithmetic interpret one another, with uniform finite-proof transformations yielding a PA-provable consistency equivalence.

The new lower proof obtains induction before successor existence. Global least numerical counterexamples then eliminate all free numerical parameters from bounded constructions. This avoids the earlier power-source recovery and stored-parameter machinery. The closed primitive shell remains 20 nodes; the improvement is matrix freedom and proof structure, not a claimed 19-node shell. The version allowing only closed matrices has a two-object model, making one free matrix role sharp within this architecture and this internal calibration.

For contradiction search we combine the inherited positive target and arithmetic sharing with a new bijective dyadic codec. A single division-free polynomial emitter uses nine nonlinear multiplications, has degree 128 and leading coefficient 2^{71}, and accepts every natural input. A complete eight-case theorem stream searches for the atom of code zero. The archive includes executable reference code and targeted independent checks. The unbounded interpretation proofs are conventional mathematics; no new literal TM state count or proof-assistant certificate is asserted.

Contents

1 The proposed theory and the actual improvement 3

2 Numerical induction before successor existence 5

3 Global least failures remove the source parameter 8

4 Saturated graphs and arithmetic 10

5 A strict reduction in original-language strength 15

6 The upper interpretation and internal proof transport 16

7 A positive unary schema and a fixed atomic target 17

8 A division-free emitter with no rejected inputs 19

9 A complete total theorem stream 22

10 Evidence, scope, and further research questions 25

1 The proposed theory and the actual improvement

Work in classical first-order logic over nonempty domains, with a single binary relation E, written $x \in a$. There is no logical equality, constant, or function symbol. The notation

$$x \subseteq a \quad :\Longleftrightarrow \quad \mathrm{Sub}(x,a) := \forall t\,(t \in x \to t \in a)$$

is an abbreviation. For each formula φ satisfying $\mathrm{FV}(\varphi) \subseteq \{x\}$, take the axiom

$$\boxed{\begin{aligned}&\forall a\,\exists b\,[\,\forall x\,(x \in b \to \varphi(x))\\ &\qquad\qquad \wedge \forall x\,(x \subseteq a \to (\varphi(x) \to x \in b))\,].\end{aligned}} \tag{U}$$

These are all the nonlogical axioms of U. Bound variables are freshened, and quantifier complexity is unrestricted. In particular, the source a cannot occur freely inside the matrix. This is a restriction to one free matrix variable, not to one-variable logic.

For the definable class X selected by φ, the informal content is

$$X \cap \mathcal{P}(a) \;\subseteq\; b \;\subseteq\; X.$$

Here $\mathcal{P}(a)$ denotes represented subsets for explanatory purposes; its existence as an object is not an axiom. The cover may contain additional members of X.

Theorem 1.1 (Main result)**.** *There are interpretations of full first-order* PA *in* U *and of* U *in* PA*. The finite refutation transformations can be chosen primitive recursive, with correctness provable in PA. For the corresponding ordinary arithmetizations,*

$$\mathsf{PA} \vdash \mathrm{Con}(\mathsf{U}) \leftrightarrow \mathrm{Con}(\mathsf{PA}).$$

The proof occupies the next four sections. Its new step is to derive numerical induction while successor is only a partially available relation. Once induction is obtained, a least failing numerical tuple can be quantified inside a unary matrix. This proves exactly the bounded constructions needed for arithmetic.

The classical exactification trick from the source-only drafts is unavailable: replacing $\varphi(x)$ by $\mathrm{Sub}(x,a) \wedge \varphi(x)$ would introduce the forbidden free source. The proof never uses that substitution. It does not derive unrestricted separation, global extensionality, arbitrary ambient powers, or a common bound for arbitrary ambient objects.

1.1 A precise count and a limited sharpness statement

Put

$$U_\varphi(b) := \forall x(x \in b \to \varphi(x)), \qquad V_\varphi(b,a) := \forall x(\mathrm{Sub}(x,a) \to (\varphi(x) \to x \in b)).$$

In the basis $\{E, \forall, \to, \bot\}$, a closed primitive instance is

$$\forall a\big((\forall b\,[U_\varphi(b) \to (V_\varphi(b,a) \to \bot)]) \to \bot\big). \tag{1}$$

Counting one node for each constructor and an opaque matrix occurrence gives

$$4E + 5\forall + 7\to + 2\bot + 2\varphi = 20.$$

For a matrix of m nodes the exact tree size is $18 + 2m$. Variable labels, numeral encodings, sharing, proof rules, and the source generator are excluded from this particular count. The previous source-only schema has the same count. A source-open presentation has $17 + 2m$ for either theory, and that inherited convention is not a new saving.

Proposition 1.2 (Zero free roles collapse)**.** *Restricting the same schema to closed matrices produces a theory with a two-object model whose consistency is provable in PA.*

Proof. Let the objects be e, u, with no members of e and with both e, u members of u. For any closed matrix φ, its truth value is independent of x and the source. If it is false, choose $b = e$; if it is true, choose $b = u$. Both clauses hold at every source. Evaluation in this fixed finite structure is primitive recursive, and PA verifies satisfaction of every axiom and soundness of finite derivations. Thus PA proves this restricted theory consistent. □

If PA is consistent, the zero-role version therefore cannot have a PA-provable consistency biconditional with PA, by the second incompleteness theorem. One free role is sharp for this precise schema family and calibration. This does not prove an absolute lower bound on axiom size or on Turing-machine states.

1.2 Review of the eight supplied drafts

Table 1 distinguishes the eight packages actually attached here from the older collections discussed inside them. Complete main manuscripts and their relevant included proof sections were reviewed. Their strongest overlapping conclusions are credited as inherited. Input file hashes are in `sources/input_manifest.json`.

Table 1: The eight supplied packages and what this iteration uses.

ID	Packet	Contribution and relation to this result
D1	LP20 Source Only Power Filter	Already proves the closed 20-node two-slot theory through a hereditary saturated core, recoverable powers, and numerical storage. Also distinguishes consistency strength from ambient bounded-pair consequences.
D2	`pa_local_falsity`	Positive local falsity, an atomic target, useful collisions at input codes 0 and 1, and total raw numeric generation. Its radix-four atom-alias variant is already present.
D3	`pa_power_filter_synthesis_2026-10-27`	Bounded parameter compilation, source frames, and exact comparison of several stronger interfaces. Its source-independent one-payload family still needs one additional fixed instance; U removes that dependency.
D4	`pa_power_synthesis`	An existing nine-pairing ordinary emitter, clause-order polynomial optimization, and a complete total theorem stream. Nine pairings alone are therefore not new here.
D5	`pa_source_only`	Source-only calibration using saturated numerical representatives and least failing bounded tuples; direct witness tableaux. Its arithmetic backend is the main ancestor of the shorter lower proof here.
D6	`pa_source_only(1)`	Another complete source-only lower interpretation and an explicit Hilbert proof search. Source recovery and transitive storage solve the parameter issue at two free slots.

ID	Packet	Contribution and relation to this result
D7	`pa_source_only_2026-10-27`	Source-only calibration through a saturated extensional domain; a frozen open emitter with careful freshness and padding.
D8	`pa_source_only_2026-10-27(1)`	Source-only calibration and a positive diagonal-target interface. Its unrestricted positive builder has 14 constructors and uses a fresh-label increment.

The main new mathematical contribution relative to these eight drafts is the unary theorem, with induction before successor existence and globally least failed numerical constructions. The numerical contribution combines earlier sharing and local-falsity ideas with a bijective dyadic grammar, a fixed minimal guard within the inspected orientation family, and a division-free three-register arithmetic schedule. Historical priority beyond this comparison is not claimed.

Compact finite-set presentations of PA have established precedents. Friedman's discussion of simplicity already records a small filtered-adjunction theory mutually interpretable with PA [9, Section C2]; Kaye and Wong analyze the distinctions among finite-set interpretations [10]. The new claim here concerns the precise unary equality-free schema (U), for which the entire lower proof is supplied.

1.3 The 215 versus 238 motivation

The inspected K215 repository reports a checked internal PA theorem about the literal 215-state machine and ZFC refutations [11]. The linked ProveIt verification separately reports the external Lean/Rocq endpoints [13]. The inspected PA238 development proves the literal 238-state blank-tape halting equivalence with finite ordinary PA refutations [14]. These source statements were read; their large formal builds were not rerun here. No exhaustive current-record survey is asserted.

Consistency strength does not impose an ordering on the lengths of descriptions of particular proof searchers. A stronger theory can have a shorter convenient schema, and the surrounding logic, variable coding, proof enumeration, initialization, and compiler can dominate the final transition table. Theory K exemplifies the value of optimizing that whole presentation [12]. Our mathematical and numerical improvements therefore remain separate from any future compiled state count.

2 Numerical induction before successor existence

The numerical representatives and saturated codes below are adapted from the supplied source-only draft [5]. The new dependency order is essential: we prove induction and global minimization before constructing successors. This makes a source-free unary matrix sufficient.

Throughout this section membership is the sole primitive binary relation. Define

$$\begin{aligned} u \simeq v &:\Longleftrightarrow \forall z\,(z \in u \leftrightarrow z \in v),\\ \mathrm{Sat}(a) &:\Longleftrightarrow \forall u \forall v\,[u \simeq v \to (u \in a \leftrightarrow v \in a)],\\ \mathrm{Tr}(a) &:\Longleftrightarrow \forall u \forall v\,[(u \in v \wedge v \in a) \to u \in a]. \end{aligned}$$

The relation $\simeq$ is an equivalence relation by logic alone. It is always a congruence in the second coordinate of membership. In the first coordinate it is a congruence when the containing object is

saturated. We never assume that it is a congruence throughout the ambient universe.

Let $\mathrm{Lin}(a)$ say that membership among the members of a is irreflexive and transitive, and that

$$\forall u \in a\, \forall v \in a \quad (u \simeq v \vee u \in v \vee v \in u).$$

Here and below bounded quantifiers abbreviate ordinary first-order formulas. When the members are saturated, this is a strict linear order on their $\simeq$-classes. Define $\mathrm{EP}(s,t)$ to say that the intersection of their member profiles, when nonempty, has endpoints:

$$\begin{aligned}(\exists z\,(z \in s \wedge z \in t)) \;\rightarrow\; \exists l \exists g\,[&l \in s \wedge l \in t \wedge g \in s \wedge g \in t \\ &\wedge \forall z\,((z \in s \wedge z \in t) \rightarrow ((l \simeq z \vee l \in z) \wedge (z \simeq g \vee z \in g)))].\end{aligned}$$

No intersection object is asserted to exist. The numerical predicate is the following single formula:

$$\begin{aligned}\mathsf{N}(a) :\Longleftrightarrow\; &\mathrm{Sat}(a) \wedge \mathrm{Tr}(a) \wedge \mathrm{Lin}(a) \wedge \forall u\,(u \in a \rightarrow \mathrm{Sat}(u)) \\ &\wedge \forall s \forall t\,[(t \simeq a \vee t \in a) \rightarrow \mathrm{EP}(s,t)].\end{aligned} \tag{2}$$

Its quantifiers range over existing objects, including objects in nonstandard models; it asserts neither external finiteness nor external well-foundedness. Finally put

$$\mathrm{Emp}(a) := \forall z\; z \notin a, \qquad \mathrm{S}(a,b) := \forall z\,[z \in b \leftrightarrow (z \in a \vee z \simeq a)].$$

Thus a successor, when it exists, adds the entire coextensionality class of its predecessor.

Lemma 2.1 (Local facts without successor totality)**.** *Empty objects exist and satisfy* N*. The predicates* Sat *and* N *are invariant under* $\simeq$*. For numerical* a*:*

1. $a \notin a$*, and every member of* a *is numerical;*
2. *if* $\mathrm{S}(a,b)$ *holds, then* b *is numerical and* a *is its greatest member class;*
3. *if* a *is nonempty, it has a greatest member* d*, unique modulo* $\simeq$*, and* $\mathrm{S}(d,a)$*; its least members are empty.*

The relation S *is invariant under* $\simeq$ *in its two arguments. On numerical classes its existing outputs are unique, its existing predecessors are unique, and no existing successor is empty. None of these assertions presupposes that every number has a successor.*

Proof. The unary false-matrix instance of U has an output with no members. An empty object satisfies every clause of (2). Saturation and numericality depend only on the member profile of their argument, so are $\simeq$-invariant.

If $a \in a$, irreflexivity in $\mathrm{Lin}(a)$ applies to a itself, giving a contradiction. Let $i \in a$. Then $\mathrm{Sat}(i)$ is part of $\mathsf{N}(a)$, and transitivity gives $i \subseteq a$. If $u \in v \in i$, all three of u, v, i belong to a; transitivity of the restricted order on a gives $u \in i$. Hence i is transitive. Its linear order and saturation of its members are inherited. For an endpoint cutoff t, if $t \in i$, transitivity puts t in a; if $t \simeq i$, saturation of a also puts t in a. The endpoint clause for a therefore proves the required clause for i.

Suppose $\mathrm{S}(a,b)$. Its member profile consists of the old members of a and one new $\simeq$-class, that of a. This profile is saturated. Every old member is saturated, and any new member is coextensional with the saturated object a. The profile is transitive: a member of an old member lies in a, and a member of an object coextensional with a also lies in a. Every old member lies below every new representative. There is no reverse edge, because an edge from a representative of $[a]$ into an old

member, followed by transitivity and saturation, would force $a \in a$. There is no edge within the new class for the same reason. Thus the old strict class order extends by its one greatest class.

For the endpoint condition of b, a cutoff $t \in b$ is either an old member or coextensional with a, so the condition for a applies. If $t \simeq b$, its intersection with any s is the old intersection with a, possibly augmented by representatives of $[a]$. If the old part is nonempty, its least endpoint remains least, and its greatest remains greatest unless a new representative is present. If the old part is empty, any present new representative is both endpoints. This proves $\mathsf{N}(b)$ and the greatest-class assertion.

For nonempty a, apply its endpoint clause with $s = t = a$ to obtain a greatest member d. It is numerical by heredity. Greatestness gives $z \in a \to (z \in d \vee z \simeq d)$. Conversely, transitivity gives $z \in d \to z \in a$, and saturation of a gives $z \simeq d \to z \in a$. This is $\mathrm{S}(d, a)$. If a least member l had a member z, then $z \in a$, and leastness would give $l \simeq z$ or $l \in z$, each contradicting the strict class order since $z \in l$. Thus l is empty.

The definition of S proves its invariance and uniqueness of output profiles. Each numerical predecessor is the greatest member class of its output, so predecessors are unique as well. Every successor contains its predecessor and is therefore nonempty. □

Theorem 2.2 (Full representative induction before total successors)**.** *For every fixed ambient formula $\theta(v, \bar{p})$, the theory U proves the following induction implication, with arbitrary parameters $\bar{p}$:*

$$\begin{aligned}[\forall e\,(\mathrm{Emp}(e) \to \theta(e, \bar{p})) \\ \wedge\, \forall a \forall b\,((\mathsf{N}(a) \wedge \mathrm{S}(a, b) \wedge \theta(a, \bar{p})) \to \theta(b, \bar{p}))] \\ \to \forall n\,(\mathsf{N}(n) \to \theta(n, \bar{p})).\end{aligned} \tag{3}$$

No invariance of θ under $\simeq$ is required.

Proof. First let $Q(v)$ have no other free variables. Assume its base and step premises, and suppose $\mathsf{N}(n) \wedge \neg Q(n)$. Apply the legal unary instance with matrix $\neg Q(x)$ and source n, obtaining B. For every $x \in n$, transitivity gives $x \subseteq n$, so the two weak filter clauses imply

$$x \in B \quad \longleftrightarrow \quad \neg Q(x) \qquad (x \in n). \tag{4}$$

If the intersection of B with n is nonempty, take a least member i. It is numerical by heredity and fails Q by the upper filter clause. It is not empty, by the base premise. Let d be its greatest member; Lemma 2.1 gives $\mathsf{N}(d)$ and $\mathrm{S}(d, i)$. Transitivity puts d in n. If $\neg Q(d)$, then (4) puts d in the same intersection, contrary to its least class $[i]$, since $d \in i$. Hence $Q(d)$, and the step premise gives the contradiction $Q(i)$.

If that intersection is empty, every member of n satisfies Q. The counterexample n is nonempty, so its greatest member d satisfies $Q(d)$ and $\mathrm{S}(d, n)$. The step premise again contradicts $\neg Q(n)$. This second case is the crucial change: it uses n itself as the least failure and does not require an already existing successor of n.

For a formula with parameters, let $A_\theta(\bar{p})$ be the conjunction of its base and step premises in (3). Form the unary formula

$$Q(v) := \forall \bar{p}\,[A_\theta(\bar{p}) \to \theta(v, \bar{p})],$$

freshening every bound variable. Its base follows from the base component of A_θ. Given an existing successor and Q at its predecessor, fix any parameters satisfying A_θ; the step component then gives Q at the successor. The unary result proves Q for every number. Specializing the bound parameters gives (3). The filter matrix used in this proof still has only its tested object free. □

3 Global least failures remove the source parameter

For numerical arguments write

$$a \preccurlyeq b \quad :\Longleftrightarrow \quad (a \simeq b \vee a \in b).$$

Lemma 3.1 (Global comparison without successor totality)**.** *For any two numbers a, b, exactly one of $a \simeq b$, $a \in b$, and $b \in a$ holds. This conclusion can be proved before totality of* S*.*

Proof. First prove by Theorem 2.2, on b, the conditional no-gap property

$$\mathsf{N}(a) \wedge \mathsf{N}(a') \wedge a \in b \wedge \mathrm{S}(a, a') \quad \to \quad (a' \in b \vee a' \simeq b). \tag{5}$$

The empty case is vacuous. At an existing successor $\mathrm{S}(b, b')$, membership $a \in b'$ means $a \in b$ or $a \simeq b$. In the first case, the induction hypothesis puts a' in b or in $[b]$, so in b'. In the second case, successor congruence gives $a' \simeq b'$. This uses existing successors only.

Now induct on a in the assertion that it compares with every number b. An empty a is coextensional with any empty b. If b is nonempty, its least member is empty; saturation of b therefore gives $a \in b$. For the step $\mathrm{S}(a, a')$, compare a with b by the induction hypothesis. If $b \in a$ or $b \simeq a$, then $b \in a'$. If $a \in b$, apply (5) to obtain $a' \in b$ or $a' \simeq b$. This proves exhaustiveness. Irreflexivity, transitivity, and saturation on numbers make the alternatives disjoint. Consequently the numerical classes are strictly linearly ordered as claimed. □

Lemma 3.2 (Global least numerical tuples)**.** *Every fixed formula $P(v, \bar{q})$ invariant under $\simeq$ in its numerical argument has a least numerical class whenever it has a numerical witness. The parameters $\bar{q}$ may be arbitrary objects. More generally, for each fixed positive integer k, every formula $B(n_1, \ldots, n_k)$ invariant in its numerical arguments has a lexicographically least numerical tuple whenever it has one. Its least tuples are unique coordinatewise modulo $\simeq$ and are described by a fixed first-order formula. If B has no additional free parameter, neither does this least-tuple formula.*

Proof. For the first claim, apply full representative induction to the assertion that the numerical class segment $I_n = \{i : \mathsf{N}(i) \wedge i \preccurlyeq n\}$ either has no P-member or has a least one. At an empty n, the segment consists of its one class, and invariance of P handles all representatives. At an existing successor $\mathrm{S}(n, n')$, the definition gives $I_{n'} = I_n \cup [n']$. If an old least witness exists it remains least; otherwise the new class either supplies the least witness or supplies none. This proves the induction assertion without totality of successors. If P has witness n, its least witness in I_n is globally least: comparison shows every number outside I_n is larger than n.

For tuples, minimize the first coordinate for which an extension to a B-tuple exists. Keep a chosen representative as a parameter of the derived least-number theorem, and minimize the second coordinate among its extensions; continue a fixed k times. Invariance transports extensions between equivalent choices of previous coordinates. The result is a lexicographically least tuple and is unique coordinatewise modulo $\simeq$.

For precision, let $\bar{r} <_{\mathrm{lex}} \bar{s}$ abbreviate the finite disjunction

$$\bigvee_{j=1}^{k} \left[\left(\bigwedge_{i<j} r_i \simeq s_i \right) \wedge r_j \in s_j \right].$$

One may take

$$L_B(\bar{r}) :\Longleftrightarrow \bigwedge_i \mathsf{N}(r_i) \wedge B(\bar{r}) \\ \wedge \forall \bar{s} \left[\left(\bigwedge_i \mathsf{N}(s_i) \wedge B(\bar{s}) \right) \to \left(\bigwedge_i r_i \simeq s_i \ \vee \ \bar{r} <_{\text{lex}} \bar{s} \right) \right]. \tag{6}$$

Tuple notation here abbreviates a fixed finite list of variables, not objects coding tuples. Formula (6) and its proof are obtained uniformly from the syntax of B. □

Theorem 3.3 (Bounded numerical comprehension)**.** *Let $\chi(z, \bar{n})$ be any fixed formula whose only free variables are the displayed ones, with k numerical parameter positions. Let I_χ assert invariance of χ when those parameters are replaced by coextensional numerical representatives, and let*

$$S_\chi : \quad \forall \bar{n} \in \mathsf{N} \, \exists a \, \forall z \, (\chi(z, \bar{n}) \to z \subseteq a).$$

Then U proves the conditional theorem

$$(I_\chi \wedge S_\chi) \ \to \ \forall \bar{n} \in \mathsf{N} \, \exists b \, \forall z \, (z \in b \leftrightarrow \chi(z, \bar{n})). \tag{7}$$

No invariance condition is imposed on the tested object z.

Proof. Assume I_χ and S_χ. For $k > 0$, suppose an output fails to exist at some numerical tuple. Define

$$B(\bar{n}) := \neg \exists b \, \forall z \, (z \in b \leftrightarrow \chi(z, \bar{n})).$$

The assumed invariance of χ makes B invariant in its numerical arguments. Lemma 3.2 supplies a globally least failing tuple $\bar{r}$. Use the following legal unary matrix:

$$\varphi(z) := \exists \bar{n} \, [L_B(\bar{n}) \wedge \chi(z, \bar{n})]. \tag{8}$$

Every tuple satisfying L_B is coordinatewise coextensional with $\bar{r}$, so I_χ proves $\varphi(z) \leftrightarrow \chi(z, \bar{r})$ for every ambient object z. Choose a support source a from S_χ at $\bar{r}$. Apply the unary weak filter to (8) at that source. Its upper clause gives $z \in b \to \chi(z, \bar{r})$. Conversely, $\chi(z, \bar{r})$ implies both $z \subseteq a$ and $\varphi(z)$, so the lower clause gives $z \in b$. This is the forbidden exact output.

For $k = 0$, the formula $\chi(z)$ is already unary. Choose any supporting source from S_χ and apply its weak-filter instance directly; the same two implications give exactness. □

The support source is an existentially chosen object in the proof; it does not occur free in (8). The argument proves a derived conditional comprehension theorem. It neither exactifies all source-free instances nor licenses arbitrary ambient parameters. A use of the theorem must supply proofs of its stated invariance and support premises. The least-failure predicate is a finite formula formed from χ; no algorithm evaluates it, and no self-referential truth predicate is introduced.

Corollary 3.4 (The bootstrap is complete)**.** *Every number has a numerical successor. Every number n has an exact power witness*

$$\text{Pow}(n, W) := \forall z \, (z \in W \leftrightarrow z \subseteq n).$$

Every finite tuple of numbers is contained in a common numerical object.

Proof. Apply Theorem 3.3 first to $\chi(z, n) := (z \in n \vee z \simeq n)$. It is invariant in n. Every selected object is a subset of n, by transitivity or coextensionality, so n itself is a support source. Its exact output satisfies $\mathrm{S}(n, b)$ and is numerical by Lemma 2.1.

Next use $\chi(z, n) := (z \subseteq n)$, again invariant in n and supported at n. The output is the asserted exact power. Only powers of numerical sources have been proved here; no power axiom for all ambient objects is being assumed.

Finally, numerical comparison chooses a greatest class from any fixed finite tuple. A successor of its representative contains every entry of that tuple, by saturation and the successor profile. □

The order of these arguments rules out the tempting circular proof: local predecessor facts imply induction; induction implies comparison and least failures; least failures imply bounded numerical comprehension; only then does comprehension imply total successors and numerical powers. Neither a preliminary extensional core nor a source-recovery decoder is needed.

4 Saturated graphs and arithmetic

We now use bounded numerical comprehension to recover the graph constructions needed for arithmetic. The all-representatives coding and the canonical history construction are inherited from draft [5]; their existence premises now follow from the unary theory. No graph object is used as a free parameter of a filter matrix.

4.1 Singletons, pairs, and ordered-pair codes

Define the following fixed formulas:

$$\begin{aligned} \mathrm{SSing}(h, x) &:\Longleftrightarrow \forall z\,(z \in h \leftrightarrow z \simeq x), \\ \mathrm{SPair}(d, x, y) &:\Longleftrightarrow \forall z\,(z \in d \leftrightarrow (z \simeq x \vee z \simeq y)), \\ \mathrm{OP}(p, x, y) &:\Longleftrightarrow \mathsf{N}(x) \wedge \mathsf{N}(y) \wedge \forall u\,[u \in p \leftrightarrow (\mathrm{SSing}(u, x) \vee \mathrm{SPair}(u, x, y))]. \end{aligned}$$

The last construction collects every witness for the two Kuratowski component profiles, not an arbitrarily chosen representative of each.

Lemma 4.1 (Saturated numerical codes)**.** *Every number has an* SSing *witness, and every pair of numbers has* SPair *and* OP *witnesses. These formulas are invariant under $\simeq$ in all their arguments, and their outputs are saturated and unique modulo $\simeq$. An* OP *code determines both numerical coordinates uniquely modulo $\simeq$. If $x, y \in n$, $\mathsf{N}(n)$, and $\mathrm{Pow}(n, W)$, then every $\mathrm{OP}(p, x, y)$ satisfies $p \subseteq W$.*

Proof. For a numerical x, use Theorem 3.3 on the target $z \simeq x$, supported at x. For numerical x, y, choose a common number n containing both, using Corollary 3.4. The target $z \simeq x \vee z \simeq y$ is invariant in its numerical parameters and is supported at n: its selected objects are coextensional with members of the transitive object n. This gives SPair.

If $x, y \in n$, saturation of n shows that every singleton or pair witness for those coordinates is a subset of n. Therefore the target

$$\mathrm{SSing}(u, x) \vee \mathrm{SPair}(u, x, y)$$

is supported at n. It is invariant in the numerical parameters x, y. Another application of Theorem 3.3 gives the OP witness.

Each output profile is unchanged when its inputs are changed within their coextensionality classes. Its membership test is invariant in the tested object as well, which proves saturation. The displayed exact profiles give output uniqueness modulo $\simeq$.

For decoding, an OP witness satisfies

$$\forall u\,(u \in p \to z \in u) \quad \longleftrightarrow \quad z \simeq x, \tag{9}$$

$$\exists u\,(u \in p \wedge z \in u) \quad \longleftrightarrow \quad (z \simeq x \vee z \simeq y). \tag{10}$$

A singleton witness exists and belongs to p, so it forces the right-hand condition of (9); every possible component contains that entire class, proving the converse. A pair witness exists and belongs to p, proving both directions of (10). The first formula determines $[x]$ and the second determines $\{[x], [y]\}$; together they determine $[y]$, including the case $[x] = [y]$.

Finally, every $u \in p$ is a singleton or pair witness and hence satisfies $u \subseteq n$. The power identity puts u in W. This is precisely $p \subseteq W$. □

4.2 Comprehension for numerically parameterized graphs

For any object G put

$$V_G(x, y) := \exists p\,(p \in G \wedge \mathrm{OP}(p, x, y)).$$

The numerical coordinates of this relation are invariant under $\simeq$, even if G is an arbitrary ambient object. Replacing G by a coextensional object preserves the relation as well.

Theorem 4.2 (Numerical graph comprehension)**.** *Let $R(x, y, n, \bar{r})$ be any fixed formula whose only free variables are the displayed numerical ones, and suppose its invariance under $\simeq$ in all those arguments has been proved. Then for every numerical $n, \bar{r}$ there exists a saturated G such that*

$$p \in G \quad \longleftrightarrow \quad \exists x \exists y\,[x \in n \wedge y \in n \wedge \mathrm{OP}(p, x, y) \wedge R(x, y, n, \bar{r})]. \tag{11}$$

Its decoded relation satisfies

$$V_G(x, y) \quad \longleftrightarrow \quad x \in n \wedge y \in n \wedge R(x, y, n, \bar{r}). \tag{12}$$

The parameters $\bar{r}$ need not themselves belong to n.

Proof. The right side of (11), viewed as a predicate of p with numerical parameter tuple $(n, \bar{r})$, is invariant in those parameters. Coextensionality of n preserves its member profile, and the other assertions are invariant by hypothesis and Lemma 4.1. Choose an exact power W of the number n, using Corollary 3.4. Every selected code is a subset of W, by Lemma 4.1. Thus Theorem 3.3 supplies the exact output G. The membership test depends only on the profile of p, so G is saturated.

For the reverse implication in (12), coordinates belonging to n are numerical by heredity, and an ordered-pair code exists; (11) puts it into G. Conversely, a code witnessing $V_G(x, y)$ is selected by some coordinates $x', y' \in n$ satisfying R. Unique decoding gives $x \simeq x'$ and $y \simeq y'$. Saturation puts x, y in n, and invariance of R proves the desired right side. □

Only a power of a numerical source is needed here. The source-free proof uses neither iterated powers nor a formula recovering a source from its power. The bound and every parameter are bound inside the least-failure matrix of Theorem 3.3.

4.3 Deterministic histories with explicit value bounds

Fix a formula $T(u, v; \bar{t})$ with only the displayed free variables. Its parameters, input, and output are numerical. Assume it is invariant under $\simeq$ and defines a total function on numerical classes:

$$\forall u \in \mathsf{N}\ \exists v \in \mathsf{N}\ T(u, v; \bar{t}),$$
$$u \simeq u' \wedge T(u, v; \bar{t}) \wedge T(u', v'; \bar{t}) \quad \to \quad v \simeq v',$$

where the numerical guards in the second line are understood. These assumptions will be proved for the particular transitions used to define addition and multiplication.

For numerical $a, b, c, d, \bar{t}$, define $H_T(f; a, b, c, d; \bar{t})$ by the following finite first-order conditions:

1. Every member of f is an OP code with numerical coordinates $i \preccurlyeq b$, $y \preccurlyeq d$.
2. Each numerical $i \preccurlyeq b$ has a decoded value $y \preccurlyeq d$, and any two values at the same index are coextensional.
3. For every empty e, $V_f(e, a)$ holds, and $V_f(b, c)$ holds.
4. If $\mathrm{S}(i, j)$, $j \preccurlyeq b$, $V_f(i, u)$ and $V_f(j, v)$, with numerical arguments, then $T(u, v; \bar{t})$.

The literal formula also includes N-guards on every free numerical argument. The bound d is separate from the terminal value c; no monotonicity of T is assumed. The definition is invariant in every numerical argument, directly from the invariance of comparison, OP, and T. This fact is established before the existence or uniqueness of histories.

Lemma 4.3 (Pointwise comparison before history existence)**.** *Two T-histories with coextensional initial values have coextensional values at every index common to their domains. Their lengths and numerical value bounds may differ.*

Proof. Keep the two histories and all their parameters fixed. Apply full representative induction, Theorem 2.2, to the assertion that their values agree at i whenever i belongs to both domains. At an empty index, their initial clauses and the functionality of each graph identify its values with its initial class.

For an existing successor $\mathrm{S}(i, j)$, if j belongs to both domains, so does i, because $i \in j$ and those numerical segments are initial. Take the two values at i from graph totality. They are coextensional by the induction hypothesis. Each value at j is a T-output of the corresponding prior value, so functionality of T makes those new values coextensional. The arbitrary graph objects occur only as parameters of the already derived induction theorem. No graph restriction, graph object coding, or filtering with a free graph parameter is used. □

Theorem 4.4 (Existence of bounded numerical histories)**.** *For every numerical initial value a and length b, there are numerical c, d and an object f satisfying $H_T(f; a, b, c, d; \bar{t})$.*

Proof. Induct on b with the numerical parameters $a, \bar{t}$ retained. At an empty b, choose a number n containing $a, b, \bar{t}$ and use Theorem 4.2 with

$$R(i, y) := (i \simeq b \wedge y \simeq a).$$

This supplies a history on the single index class $[b]$, with terminal value and bound a. No successor has an empty output, so there are no transition conditions between indices in this domain.

For the step, let $\mathrm{S}(b, b')$. Take an old history with terminal value c and bound d, and choose c' satisfying $T(c, c'; \bar{t})$. Comparison supplies a numerical d' at least d and c'. Choose a number n containing the finite numerical tuple $a, b, b', c, c', d, d', \bar{t}$. Apply Theorem 4.2 to

$$\begin{aligned} R(i, y; a, b, b', c, c', d, \bar{t}) := \big(i \preccurlyeq b \wedge \exists h\, [H_T(h; a, b, c, d; \bar{t}) \wedge V_h(i, y)]\big) \\ \vee\, (i \simeq b' \wedge y \simeq c'). \end{aligned} \tag{13}$$

Every free parameter is numerical; the old graph variable h is bound. The formula is invariant in all its numerical arguments because H_T and V_h are invariant, with the same witness h after a change of representatives.

There is at least one old history. By Lemma 4.3, all old histories in the first clause have the same values modulo $\simeq$, and those values are at most d. Thus the first clause is total and functional on I_b. The second clause adds the single value class $[c']$ at the new index class $[b']$. The domains are disjoint, and together they equal $I_{b'}$. All relevant indices and values belong to n, so the numerical graph comprehension loses none.

The resulting graph retains the initial value and has terminal value c' and bound d'. An old transition agrees with the corresponding transition in the original history by pointwise comparison and invariance of T. For a transition into the new class $[b']$, successor injectivity identifies its predecessor class as $[b]$. Its value classes are $[c]$ and $[c']$, and the transition holds by the choice of c'. This proves the induction step. □

Put

$$A_T(a, b, c; \bar{t}) := \exists f \exists d\, [\mathsf{N}(d) \wedge H_T(f; a, b, c, d; \bar{t})],$$

with numerical guards on all displayed arithmetic arguments.

Lemma 4.5 (Exact recursion equations)**.** *The relation A_T is total and functional modulo $\simeq$, and invariant in all numerical arguments. It satisfies*

$$\mathrm{Emp}(b) \;\to\; [A_T(a, b, c; \bar{t}) \leftrightarrow c \simeq a], \tag{14}$$

$$\mathrm{S}(b, b') \;\to\; \big[A_T(a, b', c'; \bar{t}) \leftrightarrow \exists c\, (\mathsf{N}(c) \wedge A_T(a, b, c; \bar{t}) \wedge T(c, c'; \bar{t}))\big]. \tag{15}$$

Proof. Totality is Theorem 4.4. Pointwise comparison at the terminal index proves functionality, and the defining formulas prove invariance. The initial and terminal clauses yield the forward zero equation; the base construction, followed by invariance, yields its reverse direction.

For the forward successor equation, take a history on $I_{b'}$ with terminal value c' and some numerical bound d. Its value at b gives a number c, and its last transition gives $T(c, c'; \bar{t})$. To obtain an actual history on I_b, choose a number containing $a, b, b', c', d, \bar{t}$ and use numerical graph comprehension with

$$i \preccurlyeq b \;\wedge\; \exists h\, [H_T(h; a, b', c', d; \bar{t}) \wedge V_h(i, y)]. \tag{16}$$

Again all free parameters are numerical. The given history witnesses totality of this relation. Pointwise comparison makes it functional and identifies all its values with the old values, which remain at most d. Its graph is an I_b-history with terminal value c. This proves $A_T(a, b, c; \bar{t})$ without separating an old graph using that graph as an illicit matrix parameter.

Conversely, given an old history and $T(c, c'; \bar{t})$, repeat the canonical extension (13) with this chosen c'. Its output supplies the desired new history. □

4.4 The interpretation of PA

Theorem 4.6 (Arithmetic in the unary theory)**.** *The theory* U *interprets full first-order Peano arithmetic.*

Proof. Use N as the interpreted domain, $\simeq$ as equality, the empty class as zero, and S as successor. The domain is nonempty. Corollary 3.4 and Lemma 2.1 prove totality and injectivity of successor, the omission of zero, and the existence of a predecessor for every nonzero class.

First take $T(u, v) := \mathrm{S}(u, v)$ and define

$$\mathrm{Add}(a, b, c) := A_T(a, b, c).$$

This transition satisfies all hypotheses of the recursion construction. Thus addition is total, invariant, and functional on numerical classes, and (14)–(15) give

$$[a] + 0 = [a], \qquad [a] + S[b] = S([a] + [b]).$$

Next, with numerical parameter a, use the already defined transition $T_a(u, v) := \mathrm{Add}(u, a, v)$ and define

$$\mathrm{Mul}(a, b, c) := \exists e\, [\mathrm{Emp}(e) \wedge A_{T_a}(e, b, c; a)].$$

Addition is already total, functional, and invariant, so this second recursion is justified. All empty representatives are coextensional; hence multiplication is total and functional and is independent of the chosen initial empty representative. Its equations are

$$[a] \cdot 0 = 0, \qquad [a] \cdot S[b] = [a] \cdot [b] + [a].$$

Translate arithmetic terms by these total functional graphs, equality by $\simeq$, and quantifiers by relativization to N. Structural induction on formulas proves invariance under replacing numerical representatives. The finite arithmetic axioms follow from the successor facts and recursion equations. Every translated induction instance, with all its parameters, is an instance of Theorem 2.2. Therefore every axiom of PA is valid under this fixed definable interpretation. □

The interpretation uses only numerical parameters in its graph comprehensions. Arbitrary graph objects occur as parameters of the already proved induction theorem or as bound variables in (13) and (16). The quotient language above describes a first-order interpretation; it does not assert the existence of a set of all numerical classes.

4.5 Uniformity needed for proof transport

The proofs above provide uniform finite derivation templates. Parameter internalization, the fixed-length least-tuple formula, the failure predicate, and the unary matrix (8) are formed by substitution and fresh bound-variable renaming. For bounded numerical comprehension, proofs of invariance and support are explicit inputs; no algorithm decides whether an arbitrary formula has those properties. Only finitely many such formulas are needed to establish the fixed arithmetic interpretation. The graph and history definitions are finite formulas formed before their existence proofs; they do not contain a satisfaction predicate or a syntactic fixed point.

After the arithmetic interpretation has been fixed, translating a PA induction instance requires only inserting its translated formula in Theorem 2.2. The remaining arithmetic axioms have fixed

finite derivations. Translation of terms to operation graphs, relativization, formula expansion, substitution, proof concatenation, and the finite loops over displayed parameter tuples are primitive recursive syntactic operations. Structural induction on formulas and induction on coded derivations verify their correctness in PA. These facts justify the usual PA-verifiable refutation transport; they do not constitute a delivered proof-assistant certificate or an implemented end-to-end proof translator.

5 A strict reduction in original-language strength

The unary restriction is a genuine weakening of the source-only schema, even though both theories have PA consistency strength. It also prevents the usual schema-level exactification. The following explicit model separates these issues.

Take two distinct empty atoms e_0, e_1, with no members, and let σ interchange them. Set $c_i = \{e_i\}$. Objects are atoms and ordinary tagged finite set objects, so atoms are distinct from all sets. Exclude the actual empty set. Extend σ recursively to finite sets, and define increasing finite levels by

$$D_0 = \{e_0, e_1, c_0, c_1\}, \qquad D_{n+1} = D_0 \ \cup \ \{S : \varnothing \neq S \subseteq D_n, \ \sigma(S) = S\}.$$

The levels are increasing: each seed remains, and an invariant set from an earlier level is still an invariant subset of the next one. Let $D = \bigcup_{n<\omega} D_n$, with membership given by the actual member sets and with atoms empty. Each level is invariant under σ, and the action is an automorphism of this structure.

Proposition 5.1 (The orbit model)**.** *The structure on D satisfies every unary weak-filter instance of* U*, but fails the source-only weak-filter schema. It also fails an exact unary power-filter instance.*

Proof. Fix a unary formula $\varphi(x)$ with no additional free variable and a source $a \in D$. The represented subsets of a form a finite set: they are the two empty atoms and the nonempty set objects whose member sets are subsets of the finite member set of a. Among them let

$$T = \{x \in D : x \subseteq_M a \text{ and } M \models \varphi(x)\}.$$

If T is empty, either atom is an empty output. Otherwise take $S = T \cup \sigma(T)$. It is a nonempty invariant finite set of elements of D, hence belongs to a sufficiently high level. Every member of S satisfies φ, since the formula is unary and σ is an automorphism. Also $T \subseteq S$. Thus S satisfies both weak filter clauses at a. This proves every instance; no satisfaction predicate is asserted inside the structure.

The object c_0 is the unique object with sole member e_0. Therefore the source-only matrix $\varphi(x, a) := (x \simeq a)$, at source $a = c_0$, requires an output having exactly the single member c_0: the upper clause excludes every other object and the lower clause includes c_0, because $c_0 \subseteq c_0$. But $\{c_0\}$ is not a seed and is not fixed by σ, so it belongs to no level.

Finally, the represented subsets of c_0 are precisely e_0, e_1, c_0. An exact true-filter instance at c_0 would require the output $\{e_0, e_1, c_0\}$. This set is neither a seed nor invariant, so is also absent. □

Every unary instance is permitted by the source-only theory, while the displayed source-only instance fails in this unary model. Consequently the unary theory is a proper deductive subtheory in the common equality-free language. The model is infinite and has unbounded finite set ranks; two

atoms does not mean two objects. The argument also shows why it would be incorrect to replace unary weak filtering by unary exact filtering without proof. The arithmetic interpretation uses the locally exact numerical outputs derived in Theorem 3.3, not such a replacement.

6 The upper interpretation and internal proof transport

6.1 Binary membership in every PA model

Interpret $E(x, a)$ by $\mathrm{Bit}(a, x)$, meaning that the xth binary digit of a is one. This is the usual arithmetic representation of hereditary finite membership; its role in finite-set interpretations is discussed in [10]. We give the particular verification needed here rather than importing a full finite-set theory.

PA proves that inclusion of the one-bits of x in those of a implies $x \leq a$. Thus the translation of $\mathrm{Sub}(x, a)$ bounds its possible solutions by a. For every fixed arithmetic formula $\chi(i, \bar{p})$, PA proves the finite-comprehension instance

$$\forall k \, \exists c < 2^k \, \forall i < k \, (\mathrm{Bit}(c, i) \leftrightarrow \chi(i, \bar{p})).$$

The proof is induction on k: at stage $k + 1$, leave the previous code unchanged if $\chi(k, \bar{p})$ is false, and add 2^k if it is true. The corresponding PA induction instance and classical excluded middle suffice. Exponentiation here denotes its PA-definable total graph; it is not an added symbol in the final interpretation.

For a given source a, apply this fact at $k = a + 1$ to

$$\chi(i, a) := \mathrm{Sub}(i, a)^* \wedge \varphi(i)^*.$$

No larger index satisfies the translated subset condition. The resulting c therefore satisfies the exact filter condition, and in particular both weak covering clauses. The argument even allows arbitrary additional matrix parameters. It establishes a formula-by-formula interpretation in every PA model, including nonstandard ones. It does not decide χ or assert a uniform truth predicate.

Also PA proves $\neg E(a, a)$ under this interpretation: the ath bit of a cannot be one, since $2^a > a$. This additional fact will validate the positive numerical interface below.

6.2 What the consistency biconditional asserts

The lower interpretation uses fixed formulas for its domain, equality, zero, successor, addition, and multiplication. Its finite non-induction obligations have the proofs in the preceding sections. For an arbitrary translated PA induction formula, the unary construction of Section 2.2 forms the universally quantified base-and-step premise and inserts it in one fixed proof pattern.

The upper interpretation expands each given unary axiom, forms its translated arithmetic predicate, and inserts that predicate into the finite-comprehension induction proof just described. Both procedures operate on finite syntax. Bound-variable renaming, substitution, construction of a fixed-length lexicographic formula, proof concatenation, and verification of earlier proof lines are primitive recursive. The least-failure argument is a classical proof pattern; the translator never searches for a semantically true least failure.

Consequently there are primitive-recursive maps f, g on proof codes such that PA proves

$$\mathrm{Prf}_{\mathsf{PA}}(p, \bot) \longrightarrow \mathrm{Prf}_{\mathsf{U}}(f(p), \bot),$$
$$\mathrm{Prf}_{\mathsf{U}}(q, \bot) \longrightarrow \mathrm{Prf}_{\mathsf{PA}}(g(q), \bot).$$

Correctness is proved by structural induction on formulas and induction through the coded finite derivation. Negating the existential versions gives Theorem 1.1. No consistency premise or reflection principle is used in these syntactic transformations.

The package proves the transformations mathematically; it does not implement a complete translator for arbitrary PA proofs or provide its Lean, Rocq, or MM0 verification. The executable proof compiler supplied later operates between two specified presentations of the positive reference calculus. These are different scopes of verification.

7 A positive unary schema and a fixed atomic target

The ordinary theory U needs only one schema. A proof-search interface can additionally remove the falsity constructor by incorporating one negative seed into the target. The local-falsity idea and atomic target are inherited from D2 [2]; the following unary calibration uses the new lower proof.

A *positive formula* here means a formula built from E, implication, and universal quantification. This is a syntactic name: antecedents of implications can have negative polarity. Let

$$D := \forall v\, E(v, v), \qquad q(a) := E(a, a).$$

For every positive unary matrix $\theta(x)$, let P_1 contain

$$\forall a \left(\left[\forall b \left(V_\theta(b, a) \to (U_\theta(b) \to q(a)) \right) \right] \to q(a) \right). \tag{P}$$

The order of the two clauses is chosen for numerical efficiency. As before, the matrix has only x free. Define the ordinary theory $L := \mathsf{P}_1 + \neg D$.

Theorem 7.1 (Positive unary calibration)**.** *The theories L and PA interpret one another, with PA-verifiable finite-proof transformations. In particular*

$$\mathsf{PA} \vdash \mathrm{Con}(L) \leftrightarrow \mathrm{Con}(\mathsf{PA}).$$

Proof. The binary-membership interpretation verifies $\neg D$ and every positive shell, using an ordinary weak-filter witness. For the lower direction, replace each falsity occurrence in an arbitrary unary matrix $\varphi(x)$ by the *closed* sentence D. Call the result φ^D. It is positive and remains unary. Under $\neg D$, structural induction gives $\varphi^D \leftrightarrow \varphi$.

At a source a with $\neg q(a)$, the positive shell is classically equivalent to the existence of a witness for its upper and lower clauses. Thus L supplies ordinary unary weak filters at every irreflexive source. Replacing falsity by $q(a)$ inside the matrix would be invalid here, because it would insert a forbidden free source; the closed D avoids that error.

The seed supplies at least one irreflexive object. False filtering there gives an empty object. Inspect the lower proof for U. Its induction filters have numerical sources, which are irreflexive by the local numerical lemma. The least-failure bounded-comprehension proof remains valid when its support premise additionally requires an irreflexive source. All applications satisfy that stronger premise: successors and numerical powers use a number; numerical singleton, pair, and ordered-pair

constructions use a number containing their coordinates; graph comprehension uses an exact power W of a number n.

Such a W is irreflexive. Indeed $n \subseteq n$ gives $n \in W$. If $W \in W$, exactness would give $W \subseteq n$, so $n \in n$, contradicting numerical irreflexivity. Therefore every actual source used by the arithmetic construction is legal. All inserted matrices are made positive by the closed substitution $\bot \mapsto D$, preserving their unary scope. The lower proof goes through in L with these explicit local support conditions. The syntax transformations are uniform as before. □

7.1 Five logical axiom families suffice

Use the positive Hilbert calculus H_+ with modus ponens, theorem generalization, and

$$\begin{aligned}
\mathsf{K}: &\quad A \to (B \to A),\\
\mathsf{S}: &\quad (A \to (B \to C)) \to ((A \to B) \to (A \to C)),\\
\mathsf{P}: &\quad ((A \to B) \to A) \to A,\\
\mathsf{UI}: &\quad (\forall v\, A) \to A[w/v],\\
\mathsf{QD}: &\quad \forall v(A \to B) \to (A \to \forall v B), \qquad v \notin \mathrm{FV}(A).
\end{aligned}$$

Substitution avoids capture. Peirce's schema P supplies classical implication. Vacuity $A \to \forall v A$ follows by generalizing $A \to A$ and applying QD when $v \notin \mathrm{FV}(A)$.

For clarity, QD also yields ordinary universal distribution. Under the two assumptions $H = \forall v(A \to B)$ and $K = \forall v A$, instantiation and modus ponens give B. Generalizing in v is permitted because v is free in neither assumption. Discharge K and H to obtain

$$\forall v(A \to B) \to (\forall v A \to \forall v B).$$

The required deduction transformation is justified directly by $\mathsf{K}, \mathsf{S}, \mathsf{QD}$: at a generalized line, generalize the already transformed implication and apply QD; its side condition is exactly the ordinary deduction-theorem restriction. Conversely, distribution plus vacuity yields QD. Thus the reduction from the six logical families in D2 to these five has a uniform syntactic justification.

7.2 Positive conservativity and the target

Put $B_* := \forall u \forall v E(u, v)$. Pure positive logic proves $B_* \to F$ for every positive formula F. Atoms follow by instantiation; the implication case follows from the induction hypothesis for the consequent; the universal case follows by generalization and QD, since B_* is closed. This constructs the proof recursively from F.

Consider the full classical calculus obtained by adding falsity and $\bot \to F$. Replacing every falsity by B_* converts a proof of a positive conclusion from positive premises to a positive proof. Each translated ex-falso axiom is replaced by the preceding derived theorem. All substitution and side conditions survive because B_* is closed. Peirce and ex falso give the usual double-negation elimination in the full calculus.

Corollary 7.2 (A single atomic target)**.** *For $q_0 = E(v_0, v_0)$, PA proves*

$$\exists p\ \mathrm{Prf}_{\mathsf{H}_+ + \mathsf{P}_1}(p, q_0) \quad \longleftrightarrow \quad \neg\, \mathrm{Con}(\mathsf{PA}).$$

Proof. A proof of q_0 generalizes to D, and the seed refutes L. Conversely discharge the closed seed from a refutation of L to obtain $\mathsf{P}_1 \vdash \neg\neg D$, hence $\mathsf{P}_1 \vdash D$. Positive conservativity removes intermediate falsity, and instantiation gives q_0. All conversions are primitive recursive and verified in PA; apply Theorem 7.1. □

The positive theory alone has a one-object total-relation model. Its ordinary consistency is not the calibrated assertion; the calibrated assertion is nonderivability of q_0, or consistency after adding $\neg D$. The positive shell has 20 schematic nodes, but its ordinary-theory presentation also has the fixed four-node seed. The search interface handles that seed through its target.

8 A division-free emitter with no rejected inputs

For machine construction it can be cheaper to accept arbitrary matrix parameters than to validate a unary restriction. The arithmetic interpretation in PA already validates the unrestricted covering schema W_∞, and W_∞ contains U. Thus unrestricted generation retains exactly the required consistency strength. The unary theorem and the following numerical implementation solve complementary optimization problems.

8.1 A bijective dyadic grammar

Let

$$\pi(i,j) := \frac{(i+j)(i+j+1)}{2} + j.$$

Use named variables $v_0, v_1, \ldots$ and the constructors

$$\begin{aligned} R(i,j) &:= 2\pi(i,j) && \text{for } E(v_i, v_j), \\ A(v,f) &:= 4\pi(f,v) + 1 && \text{for } \forall v_v \ \mathrm{dec}(f), \\ I(f,g) &:= 4\pi(f,g) + 3 && \text{for } \ \mathrm{dec}(f) \to \mathrm{dec}(g). \end{aligned}$$

Every even code is an atom. Odd codes with residue 1 modulo 4 are universal formulas; those with residue 3 are implications. These disjoint cases exhaust all natural numbers. The quantifier stores its body first and its binder second.

Lemma 8.1 (Total syntax and freshness)**.** *These constructors give a bijection between natural numbers and finite named positive formulas. Every variable label appearing in* $\mathrm{dec}(n)$ *is at most* n*, and is strictly below* n *when* $n > 0$*.*

Proof. Both coordinates of $\pi(i,j)$ are at most its value. Every recursive formula field of an implication or quantifier is therefore strictly smaller than its positive parent. Atom fields are terminal labels, including the sole zero case $R(0,0) = 0$. Strong induction proves decoding terminates and proves the label bounds. Disjoint tags and the two inverse pairing identities prove both encoder/decoder compositions are identities. □

In particular

$$\mathrm{dec}(0) = q_0, \qquad \mathrm{dec}(1) = D, \qquad \mathrm{dec}(3) = (q_0 \to q_0).$$

There is no malformed-code case, atom alias, or exceptional zero interpretation. All constructor formulas are integer polynomials: the factors 2 and 4 clear Cantor pairing's denominator.

Fix source label 0, tested label 1, and the local guard binder 2. The cached subset guard is

$$G = A(2, I(R(2,1), R(2,0))) = 1512069.$$

It requires four constructor calls to initialize or 21 bits as a bare binary literal. Those bits do not measure the cost of writing them on a blank tape. Reversing the quantifier coordinates minimizes this guard among the eight independent pairing orientations with these fixed roles; no global coding optimality is asserted.

8.2 The exact generator

For every natural n, use the input both as the matrix code and as the output-variable label:

```
e = R(1,n)
h = I(e+n,0)
u = A(1,h+4*n)
v = A(1,I(G,h+4*e))
g(n) = I(A(n,I(v,I(u,0))),0)
```

The arithmetic identities

$$I(e,n) = I(e+n,0) + 4n, \qquad I(n,e) = I(e+n,0) + 4e$$

share one pairing evaluation between the two oppositely directed implications. The resulting ledger is one R, three A, and five I evaluations: nine pairings after guard initialization.

For $n \geq 2$, the output variable is fresh by Lemma 8.1. Decoding gives precisely

$$\big[\forall v_n \, (V_\theta(v_n, v_0) \to (U_\theta(v_n) \to q_0))\big] \to q_0, \qquad \theta = \mathrm{dec}(n).$$

Here additional free matrix variables are universally intended theorem parameters. The local binder 2 occurs only inside the subset guard; it does not capture matrix occurrences. Its shadowing of the output binder when $n = 2$ is harmless. No matrix traversal, freshness scan, variable substitution, code increment, or rejected-input branch occurs in this emitter.

8.3 The two useful collisions

Let R be the theory of universal closures of all decoded $g(n)$, including $n = 0, 1$. Define

$$J := \forall x(E(x,x) \to D), \qquad \mathrm{IR} := \forall x \neg E(x,x).$$

The inherited collision idea from D2 remains valid after the coding changes:

$$\begin{aligned} &\mathrm{dec}(g(0)) \leftrightarrow (E(a,a) \vee \exists c \ \mathrm{Emp}(c)), \\ &\mathrm{dec}(g(1)) \leftrightarrow (E(a,a) \vee J), \end{aligned} \tag{17}$$

where $a = v_0$ is free outside the displayed outer wrapper.

For $n = 0$, the output binder captures the source occurrences inside its scope. A counterexample to the outer universal antecedent has the two filter clauses and a false diagonal $E(c,c)$. The upper clause then says exactly that c is empty. Conversely an empty c satisfies both clauses: the lower clause has the false matrix antecedent $E(c,c)$. The guard's binding choices therefore do not affect this identity.

For $n = 1$, the matrix is the closed D and output/test binders coincide. The upper clause is J; the lower clause is valid because $D \to E(x, x)$ is a universal instance. The outer output quantifier is vacuous, leaving $J \vee E(a, a)$. Reordering the two curried clauses preserves both identities by propositional logic.

The regular part still generates all needed positive schemas. The two missing tiny matrices have regular logically equivalent codes

$$27 = \ulcorner(q_0 \to q_0) \to q_0\urcorner, \qquad 33 = \ulcorner\forall v_2 D\urcorner.$$

All other matrix codes are already at least 2. Canonicalize source/test labels to 0 and 1, emit, then use theorem generalization, capture-avoiding instantiation, and derived alpha conversion to restore arbitrary labels. Moving other free parameters through fresh temporary names prevents collisions. These are proof transformations, not instructions executed by the emitter.

Theorem 8.2 (Raw-rule calibration)**.** *In full classical first-order logic,* $\mathsf{R} + \neg D$ *and* $\mathsf{W}_\infty + \mathrm{IR}$ *have the same deductive closure after the described naming normalization. Their proof conversions are primitive recursive and PA-verifiable. Moreover*

$$\mathsf{PA} \vdash (\exists p\ \mathrm{Prf}_{\mathsf{H}_+ + \mathsf{R}}(p, q_0)) \leftrightarrow \neg\,\mathrm{Con}(\mathsf{PA}).$$

Proof. The padding and renaming argument derives every unrestricted positive diagonal instance from R. Under $\neg D$, choose an irreflexive source a. The second collision gives J; together J and $\neg D$ imply global IR. Thus every source has false diagonal. For any ordinary unrestricted matrix, replace its falsities by the source diagonal, which is now false. This is permitted in the unrestricted interface. The positive shell supplies its ordinary weak filter. Hence $\mathsf{R} + \neg D$ proves $\mathsf{W}_\infty + \mathrm{IR}$.

Conversely $\mathsf{W}_\infty + \mathrm{IR}$ proves $\neg D$, supplies witnesses for all regular positive shells, and proves both exceptional outputs: empty objects exist and J follows from irreflexivity. This derives every axiom of $\mathsf{R} + \neg D$.

Since W_∞ contains U, the new lower interpretation gives PA strength. The binary-membership interpretation validates $\mathsf{W}_\infty + \mathrm{IR}$ in PA, giving the reverse proof transformation. Finally discharge $\neg D$ and apply the positive-conservativity argument from Section 7. Generalization/instantiation exchanges targets q_0 and D. All steps are finite uniform syntax transformations. □

8.4 One polynomial, nine nonlinear multiplications

Set $Q(s) = 2s(s + 1)$. The generator is the following polynomial in n, expressed in factored form:

```
e = n*(n+5)+2
h = Q(e+n)+3
u = Q(h+4*n+1)+5
j = h+4*e
l = Q(G+j)+4*j+3
v = Q(l+1)+5
c = Q(u)+3
b = Q(v+c)+4*c+3
z = Q(b+n)+4*n+1
g = Q(z)+3
```

It uses one multiplication for e and eight evaluations of Q. Multiplication by 2 or 4 can be addition or a binary shift. The emitter contains no division or data-dependent branch.

The degrees of $e, h, u, j, l, v, c, b, z, g$ are respectively

$$2, 4, 8, 4, 8, 16, 16, 32, 64, 128.$$

Tracking leading terms gives

$$g(n) = 2^{71} n^{128} + O(n^{127}), \qquad \log_2 g(n) = 128 \log_2 n + 71 + O(n^{-1}).$$

The leading coefficient of b is 2^{17} because its two degree-16 inputs v, c both have leading coefficient 2^7; the next two quadratic layers give 2^{35} and 2^{71}. Exact coefficient reconstruction in the supplied program confirms all 129 coefficients are positive integers.

Three persistent arithmetic registers suffice for a high-level schedule. Keep $r_0 = n$, form $r_1 = e$ and $r_2 = h$, and then execute

```
r1 = r2 + 4*r1
r2 = Q(r2 + 4*r0 + 1) + 5
r1 = Q(G + r1) + 4*r1 + 3
r1 = Q(r1 + 1) + 5
r2 = Q(r2) + 3
r1 = Q(r1 + r2) + 4*r2 + 3
r1 = Q(r1 + r0) + 4*r0 + 1
r1 = Q(r1) + 3
```

The result is $r_1 = g(n)$. Each assignment reads the old right-hand side before overwriting its destination. Multiplication and expression-evaluation workspace are excluded; this is not a three-register primitive counter-machine claim.

8.5 Fair comparisons

The raw positive emitter in D2 recomputes its n-dependent guard and uses 14 constructor pairings. Freezing the guard reduces this to 10; sharing opposite implications reduces it to nine. The longest clause is placed first in the curried pair, lowering the degree from the varying-guard baseline's 512, through 256 with a frozen upper-first order, to 128. D4 already contains nine-pairing and clause-order ideas for an ordinary falsity-bearing emitter; the new claim is their calibrated combination with raw positive collisions and the dyadic division-free grammar.

The fixed guard is 21 bits, compared with 22 in the inspected ternary comparator and 25 in its uniform radix-four comparator. These sizes exclude bootstrap machinery. A sample of 514 *matched named matrices*, re-encoded in each grammar, gives aggregate emitted-code lengths 542,344 bits for the preferred dyadic grammar, 563,166 for the ternary grammar, and 634,286 for the uniform radix-four grammar. These are diagnostic samples, not a distributional performance theorem: the dyadic output is longer in 113 ternary comparisons and 17 radix-four comparisons. It is not pointwise dominant.

No comparison of constructor counts, code lengths, or polynomial degrees establishes a TM state bound. The complete proof engine and initialization must be compiled under one common binary single-tape convention before making that claim.

9 A complete total theorem stream

An axiom emitter alone is not a contradiction searcher. The following full reference algorithm adapts D4's total-stream architecture [4] to the positive dyadic syntax and the calibrated raw rule.

Its eight branches also avoid a proof-description rejection state: an unusable inference description produces a fixed theorem.

Write $\tau = I(0,0) = 3$. Define $T(0) = \tau$. For $q > 0$, uniquely write

$$q = 1 + 8\pi(a,b) + i, \qquad 0 \le i < 8.$$

The coordinates a, b are strictly smaller than q. Set $T(q)$ by Table 2; displayed formulas denote their numeric codes.

Table 2: All eight theorem-description branches.

Tag	Kind	Output
0	K	$a \to (b \to a)$.
1	S	Unpair $b = \pi(c,d)$; output $(a \to (c \to d)) \to ((a \to c) \to (a \to d))$.
2	Peirce	$((a \to b) \to a) \to a$.
3	UI	Unpair $b = \pi(f,t)$; output $(\forall v_a\ \mathrm{dec}(f)) \to \mathrm{dec}(f)[v_t/v_a]$, using capture-avoiding substitution.
4	QD	Unpair $b = \pi(f,g)$. If $a \notin \mathrm{FV}(\mathrm{dec}(f))$, output $(\forall v_a(f \to g)) \to (f \to \forall v_a g)$; otherwise output τ.
5	AX	$g(a)$ from Section 8; b is unused.
6	MP	If $T(a)$ is literally $T(b) \to c$, output c; otherwise output τ.
7	GEN	$\forall v_a\ \mathrm{dec}(T(b))$.

Only MP and GEN consult earlier theorem descriptions. Formula arguments always decode. Variable names and formula codes are distinguished by their argument positions. Modus ponens uses literal numerical identity, which is exact named syntax because the grammar is bijective; there is no implicit semantic-equivalence oracle.

Theorem 9.1 (Soundness and exact range)**.** *T is primitive recursive, and its range is exactly the set of formula codes provable in the positive Hilbert theory with the emitted open axioms. There are primitive-recursive proof-producing and proof-compiling maps, with correctness provable in PA.*

Proof. Pairing, unpairing, formula decoding, free-variable computation, capture-avoiding substitution, and encoding are primitive recursive. Formula recursion descends to smaller codes; fresh variable names can be chosen as one more than the maximum label in the finite expressions involved. Course-of-values recursion therefore defines a primitive-recursive T.

For soundness, induct on q. The first five branches are logical axioms, except that failed QD returns τ. AX is a permitted emitted axiom. A successful MP concatenates the two already obtained finite proofs and appends modus ponens; failed MP returns τ. GEN appends generalization. The identity $F \to F$, including τ, has the familiar five-line derivation from K and S, supplied explicitly by the reference code. The induction thus constructs a finite proof $s(q)$ of $T(q)$, rather than merely asserting its semantic truth. PA verifies

$$\forall q\ \mathrm{Prf}_{\mathsf{H}_+ + \mathsf{R}}(s(q), T(q)).$$

Open emitted axioms are universally intended schemata, equivalent to their closures; they are not undischarged arbitrary assumptions. Generalization on theorems is consequently unrestricted.

Conversely, first expand any axiom-naming or alpha-conversion conventions into the specified Hilbert calculus. The emitted open axioms and their universal closures have uniform conversions by generalization and instantiation. Encode each line of a finite proof in order. K, S, Peirce, UI, and QD use their corresponding branches and the supplied formula fields. AX at input n uses description $1 + 8\pi(n, 0) + 5$. If the implication and antecedent have already compiled descriptions u, v, their MP result uses $1 + 8\pi(u, v) + 6$. Generalizing a compiled description u in label j uses $1 + 8\pi(j, u) + 7$.

Every parent description exceeds its consulted children. At a valid MP step the literal formulas match, so the default branch is not used. Induction on the proof length shows that the compiled final description has exactly its intended conclusion code. Extend the compiler to malformed proof codes by returning zero. Proof checking and these finite sequence transformations are primitive recursive, and PA verifies their conditional correctness. This proves exact range and both asserted maps. □

Corollary 9.2 (Calibrated search)**.** *Let M enumerate $q = 0, 1, 2, \ldots$, compute $T(q)$, and halt if $T(q) = 0$. Then*

$$\mathsf{PA} \vdash \mathrm{Nonhalt}(M) \leftrightarrow \mathrm{Con}(\mathsf{PA}).$$

Proof. Theorem 9.1 identifies the occurrence of zero with derivability of the positive atom q_0; Theorem 8.2 identifies that finite proof with a PA refutation. The displayed search is an ordinary effective algorithm, and its finite computation/proof conversions formalize in PA. □

This is an abstract program specification and executable reference implementation. It is not a binary single-tape transition table. Python's finite memory and recursive AST utilities impose host limits on actual runs; the theorem concerns the specified unbounded recursion. A bounded run reports `unknown` when it finds no target.

9.1 Independent certificates and failure controls

The supplied `positive_stream.py` includes a separate explicit proof checker and a compiler from its finite certificates to stream descriptions. Six identity proofs, capture-avoiding instantiation, derived vacuity, Peirce, six raw axioms, and a four-line nonlogical re-instantiation give 16 saved certificates. In the last example the source is instantiated by the output label of $g(27)$, forcing an alpha-renaming and exercising AX, GEN, UI, and MP together.

Seven invalid controls cover a false target, violated QD side condition, forward proof reference, a Boolean masquerading as a natural-number argument, an unauthorized axiom, an empty certificate, and variable capture. Tests remain active under optimized Python. The checker does not accept an unstated extension axiom to make the target derivable.

The exact target contract is separate from ordinary formula validity. The archive fixes the names and final formula codes of the supplied certificate inventory, replays their inferences, compiles each proof, and checks the resulting stream value. Checking a collection of arbitrary valid tautologies would not satisfy that release contract.

10 Evidence, scope, and further research questions

10.1 What was checked

The mathematical proof was independently reviewed at the points where an accidental parameter or circular use of successor would invalidate it. The review covered the numerical predicate, induction before totality, conditional comparison, least numerical tuples, the bounded-comprehension rule, all-representatives pair decoding, and canonical graph extension/restriction. A second independent audit challenged the same dependencies and the symmetry-model separation. These are mathematical reviews, not kernel-checked formal proofs.

The executable evidence is deliberately narrower and reproducible:

- All 66,066 binary relations on nonempty domains of size at most four were checked for the local numerical lemmas. The induction diagnostic examines 1,052,740 arbitrary unary predicates and explicitly constructs 86,748 least-failure cases: 83,238 use the empty-intersection branch and 3,510 the nonempty branch. It exhibits 83,382 base failures and 3,366 step failures. These finite structures are not claimed to satisfy the full unary schema.
- The symmetry diagnostic checks levels of sizes 4, 7, and 35. Its 224 orbit-witness cases check 15,680 upper/lower implications and 1,225 automorphism comparisons. It confirms the missing-singleton and missing-exact-power controls. Again, a finite level is not a model of the infinite theory.
- The emitter diagnostic checks 4,099 inputs in each of four coding variants, including large integers. It compares independent ASTs, round trips, variable bounds, constructor ledgers, input recovery, the polynomial form, and the register schedule.
- Both exceptional-input identities are checked on every binary relation of sizes one, two, and three, under every source assignment: 12,560 cases across the four codecs. A further 514 matched matrices are re-encoded before comparing their emitted formulas and lengths.
- Exact full coefficient arrays verify degrees 512, 256, and 128 for the inspected arithmetic variants, including the preferred leading coefficient 2^{71}. This is symbolic rational arithmetic, not fitting a polynomial to samples.
- The positive proof checker replays 16 exact named targets, tests 306 finite semantic cases for the logical conclusions, rejects seven invalid controls, and checks a 4,096-description stream prefix. Finding no target in that prefix is recorded only as a bounded implementation observation.

Normal and optimized Python runs are compared for the diagnostic suites. Saved receipts include their exact counts. The archive's replay script writes fresh results into a temporary directory and verifies the release hashes before and after execution. The final PDF was compiled from the supplied LaTeX and visually inspected. External K215, K230, and PA238 proof binaries were not rebuilt.

10.2 What is delivered

`report.tex` and `sections/` contain the complete paper; `report.pdf` is its compiled version. `code/` contains the emitter, exact polynomial checker, complete positive theorem stream, finite numerical

diagnostic, and symmetry diagnostic. `results/` contains fixed certificate targets and check receipts. Source identities and review scope are recorded in `sources/`; `SHA256SUMS` seals the release.

Run

```
python3 -B reproduce.py
python3 scripts/build_pdf.py
python3 code/positive_stream.py --search --limit 4096
```

The first command requires only Python 3.10 or later and its standard library. The second also requires a standard LaTeX installation. The third is a bounded reference search and returns `unknown` on the delivered prefix. Omitting its limit requests the unbounded semidecision search; no termination is promised.

10.3 The remaining machine question

The sharp abstract theory uses one free matrix role. The fastest route to a small machine may instead use the unrestricted polynomial emitter, because accepting every integer avoids a free-variable validator. A fair experiment must compare these alternatives within the same implementation of logic, pairing, substitution, proof enumeration, tape encoding, and blank-tape initialization.

The nine nonlinear multiplications are a concrete source-level target. They do not include decoding, proof inference, integer storage, or multiplication workspace. The 21-bit guard can be initialized arithmetically or literally; both costs belong in a compiled comparison. The total theorem stream supplies a complete semantic reference for that work, so the next machine is no longer dependent on an unspecified proof enumerator.

10.4 Further research questions

1. Can the unary lower interpretation be formalized directly in Lean, Rocq, or MM0 with the predecessor-first induction lemma as its first major theorem? This would avoid formalizing the earlier source-recovery construction.

2. Can one compile the dyadic polynomial rule and all eight proof-description branches into a literal two-symbol, one-tape machine, then prove its PA-internal halting equivalence? What state count results after initialization and arithmetic workspace are included?

3. Does the unary restriction actually reduce a compiled machine once its free-variable recognizer is counted, or does unrestricted generation remain smaller? The two mathematical calibrations permit a controlled comparison.

4. Can a different code preserve the useful 0/1 collision identities while reducing the whole substitution and proof engine, rather than only the axiom emitter? The dyadic sample already shows that one encoding need not minimize every formula.

5. Can the four-node subset guard or the split-clause wrapper be replaced by a genuinely different PA-strength construction with a smaller compiled interface? The one-role sharpness theorem addresses neither of those global questions.

6. How far can numerical bounded comprehension be generalized to non-numerical definable domains carrying a parameter-free least-class principle? The least-failure proof isolates the exact invariance and support hypotheses needed.

7. Which original-language principles separate unary weak filtering, unary exact filtering, source-only filtering, and unrestricted filtering beyond the explicit symmetry example? Their shared consistency strength does not determine their deductive closures.

The new theorem removes the last free matrix parameter while preserving a PA-provable consistency calibration. The accompanying numerical program gives a concrete next compilation target. A smaller literal machine remains an empirical and formal-verification task.

References

[1] Supplied draft D1, *A closed twenty node theory of Peano arithmetic strength: Source only power filtering through an extensional core*, `LP20_Source_Only_Power_Filter_2026-10-27`, 27 October 2026.

[2] Supplied draft D2, `pa_local_falsity`, main manuscript and included sections on local falsity, numeric frontends, arithmetic, and least failure, 27 October 2026.

[3] Supplied draft D3, `pa_power_filter_synthesis_2026-10-27`, complete manuscript and construction audits, 27 October 2026.

[4] Supplied draft D4, *Power filtering of PA strength: A sharp support bound, a polynomial axiom generator, and a complete contradiction-search algorithm*, `pa_power_synthesis`, 27 October 2026.

[5] Supplied draft D5, `pa_source_only`, main manuscript and `sections/c2_lower.tex`, 27 October 2026. Source for the saturated numerical predicate, all-representatives ordered-pair codes, and canonical deterministic-history construction adapted here.

[6] Supplied draft D6, `pa_source_only(1)/pa_source_only`, *Source-only power coverage: A two-slot theory of PA strength*, 27 October 2026.

[7] Supplied draft D7, `pa_source_only_2026-10-27/pa_source_only`, *PA from source-only power coverage*, 27 October 2026.

[8] Supplied draft D8, `pa_source_only_2026-10-27(1)/pa_source_only`, *Source-only power covering at PA strength*, 27 October 2026.

[9] Harvey M. Friedman, *Issues in the foundations of mathematics*, Gödel Lecture, 2 June 2002, Section C2, p. 10. `https://cpb-us-w2.wpmucdn.com/u.osu.edu/dist/1/1952/files/2014/01/GodelLect060202-n2gdjc.pdf`.

[10] Richard Kaye and Tin Lok Wong, *On interpretations of arithmetic and set theory*, Notre Dame Journal of Formal Logic 48(4) (2007), 497–510; expanded author version dated 15 July 2006. `https://bpb-us-w2.wpmucdn.com/blog.nus.edu.sg/dist/4/10956/files/2019/01/finitesettheory-1i2tx5t.pdf`.

[11] Elliot Glazer, *K215 halts iff ZFC is inconsistent (Metamath Zero, PA)*, README inspected 27 October 2026. `https://github.com/elliotglazer/k215-pa`.

[12] Elliot Glazer, *K230: a 230-state Turing machine for the consistency of ZFC*, README, "The theory K," inspected 27 October 2026. `https://github.com/elliotglazer/k230-rocq/blob/main/README.md#the-theory-k`.

[13] Vladimir Reshetnikov, ProveIt, *Formal correctness of the exact 215-state machine*, README inspected 27 October 2026. `https://github.com/VladimirReshetnikov/ProveIt/tree/main/Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215`.

[14] *PA238: halting and finite PA refutations*, the PA238 development in Jeffrey's busy-beaver repository; machine attributed to Mira. README and numeric-code source inspected 27 October 2026. `https://github.com/jeff87654/busy-beaver/tree/main/PA/m238`.